Luisito L. Agustin

20260313

Project 3
Roots of Polynomials

Numerical Methods for Roots of Polynomials with Real-Valued Coefficients

Luisito L. Agustin

Project 3
Roots of Polynomials

Due: start of class time, May 5, 2026

Progress Report 1	Mar. 27
Progress Report 2	Apr. 10
Progress Report 3	Apr. 17
Early Work Submission	Apr. 28

ENGG 27.01 Project

Roots of Polynomials

Luisito L. Agustin
2026 03 12

General Specifications

Use C++ to write an application that uses numerical methods to find all the real and complex roots of a polynomial with real-valued coefficients.

Detailed Specifications

Use C++ to write an application that uses numerical methods to find all the real and complex roots of a polynomial with real-valued coefficients.

The application obtains a polynomial *f(x)* by extracting its coefficients from a polynomial file compliant with the Polynomial File Format specified in a separate document. Provide a means and appropriate guidance for the user to specify the polynomial file. Do not modify any filename provided by the user. Do not assume, nor require, nor add a ".txt" extension. Use whatever filename is provided.

Use numerical methods to determine all roots of the polynomial up to the maximum precision possible. Do not use external libraries in implementing the numerical methods. Do not limit the degree of the polynomial in any way.

To provide a way to check that the roots determined are correct, write a routine to evaluate the value of the polynomial at any finite complex value. The function value of the polynomial should be 0 or very close to 0 at properly obtained estimates of the roots.

Provide a means and appropriate guidance for the user to specify the name of an output text file where the roots would be written to. Do not modify any filename provided by the user. Do not assume, nor require, nor add a ".txt" extension. Use whatever filename is provided.

All estimated roots, function values at the roots, and quadratic polynomials corresponding to complex conjugate pairs of roots will be written to an output file. The real-valued roots, real and imaginary components of complex roots, the function values at the roots, and the coefficients of the quadratic polynomial will be written to the output file in scientific notation with the number of significant figures set to the maximum precision available for `double` floating point numbers as might be determined thru the limits header of the C++ standard library.

All real-valued roots, with no imaginary component, will be written to the output file, together with evaluation of the polynomial at the roots, in the form

```
root_estimate
f ( root_estimate ) = function value
```

followed by a blank line.

Complex roots will be written to the output file in complex conjugate pairs, together with the quadratic of the form x^2 + bx + c whose roots are the complex conjugate pair, and together with evaluation of the polynomial at the roots, in the form

```
root_estimate_a+jb
f ( root_estimate_a+jb  ) = function value
root_estimate_a-jb
f ( root_estimate_a-jb  ) = function value
( x^2 + bx + c )
```

followed by a blank line.

If the number of roots is at most 20, the report sent to the output file should be sent to the console as well.

Write the routine that extracts the coefficients of a polynomial from a polynomial file as a C++ function with appropriate parameters. This function must be compliant with the Polynomial File Format. This function must not call on any other functions except those available thru the C++ standard library.

Write the routine that determines all the roots of the polynomial as a C++ function with appropriate parameters. This function should not perform any input or output. All information needed by the function must be passed to the function in appropriate data structures. The function stores all its results in appropriate data structures. This function must not call on any other functions except those available thru the C++ standard library or those that determine a single real root or a single pair of complex conjugate roots of the polynomial at a time.

Write the routine that evaluates the value of a polynomial at a complex value as a C++ function with appropriate parameters. This function should not perform any input or output. All information needed by the function must be passed to the function in appropriate data structures. The function stores all its results in appropriate data structures. This function must not call on any other functions except those available thru the C++ standard library.

Write the routines that write output to file and/or the console as C++ functions with appropriate parameters. These functions must not call on any other functions except those available thru the C++ standard library. To the extent possible, use the same function to write either to a file or the console.

Revision History - Roots of Polynomials

2019 03 08	Project 1, 2018-2, CE 33
2020 03 02	Project 1, 2019-2, CE 33 A and B
2024 10 10	required use of Polynomial File Format
2026 03 12	reorganized instructions, changed specifications for reporting results

Variable Instructions

Sections marked "(variable)" and specifications and instructions from this point onwards may vary from one use of the document to the next. These are not tracked as revisions to the exercise.

Group Work

Work on the C++ code may be done by groups. Documentation and testing must be done individually.

It is expected that C++ code submitted by groupmates may be similar or exactly the same. Points earned for the entire project will be subject to penalties for unauthorized collaboration if documentation submitted by at least two students contain at least one distinctive phrase, image, or item in common.

Submission

Implementation and Testing Environment Report

Provide details of at least one platform used in implementing and testing the project.

* name and version of compiler
* name and version of integrated development environment (IDE)
* operating system name/distribution and version

screenshot of compilation

The screenshot must show the files involved and show feedback from the IDE reporting successful compilation and linking; if the project is compiled on the command line, the screenshot must show the commands issued; if a makefile was used, include a copy of the makefile in addition to the screenshot.

screenshot of program run on the console

Copy the executable to a location whose path contains a recognizable version of your name. If working with a group, make sure that the path does not contain any version of the names of any groupmates. Create a new folder named after yourself as needed. Run the executable by typing its name at the command prompt, and take a screenshot as the executable starts. Do not run the application from within an IDE.

Basic Test

Test and document the performance of your application on the data provided:

```
POLYNOMIAL
5
1.25
-3.875
2.125
2.75
-3.5
1
This is the 5th degree polynomial x^5 - 3.5 x^4  + 2.75 x^3 + 2.125 x^2  - 3.875x +1.25
roots are 2, -1, 1 +/- 0.5i, 0.5
This polynomial appears in
[Chapra]        (book) Steven C. Chapra: Applied Numerical Methods with MATLAB for Engineers and Scientists, Int'l ed, 2005. ( Example 6.8 )
```

Obtain a screenshot of the output of the application on the console in response to basic test data. Be sure the screenshot shows the same results as those expected for the basic test. This screenshot must also show the function values when evaluated at the determined roots. Place the screenshot in a document with your name.

Do a self-evaluation using the project evaluation table provided. Place the table in a document exported to pdf.

Submit a single zip file containing:

* the pdf document containing the required screenshot
* C++ code worked on
* the self-evaluation .

Project Evaluation - Roots of Polynomials		
Item	Points	Rubrics
early work	8/8	(all or nothing) 8: at most 10% of the code in the final implementation differs from that in early work submission
implementation and testing environment report	4/4	4 - all instructions followed correctly
basic test	50/50	0 - fails basic test / does not compute the correct result from basic test data 50 - computes the correct result from basic test data and displays the correct results in the prescribed manner
	6/6	0 - no screenshot of correct basic test result 6 - screenshot of correct basic test result submitted in a pdf document, consistent with actual program run
function implementation and generality of numerical methods to determine the roots	12/12	0 - no function, or fails basic test 6 - implemented as a function with appropriate parameters 12 - generality of implementation: function is robust, efficient, works for all cases
function to evaluate value of polynomial	4/4	0 - no function 4 - function works correctly
compliant function to read polynomial files	4/4	0 - no function or not fully compliant 4 - fully compliant function
functions for producing output to files/console	4/4	0 - no function(s) or does not implement prescribed format 4 - function(s) work correctly, producing output according to the prescribed format
user interface	4/4	0 - application displays undesired/unexpected behavior or lacks some required features 4 - robust user interface with required features
self-evaluation	4/4	4 - self-evaluation accurate
total	100	

Evaluation Procedures

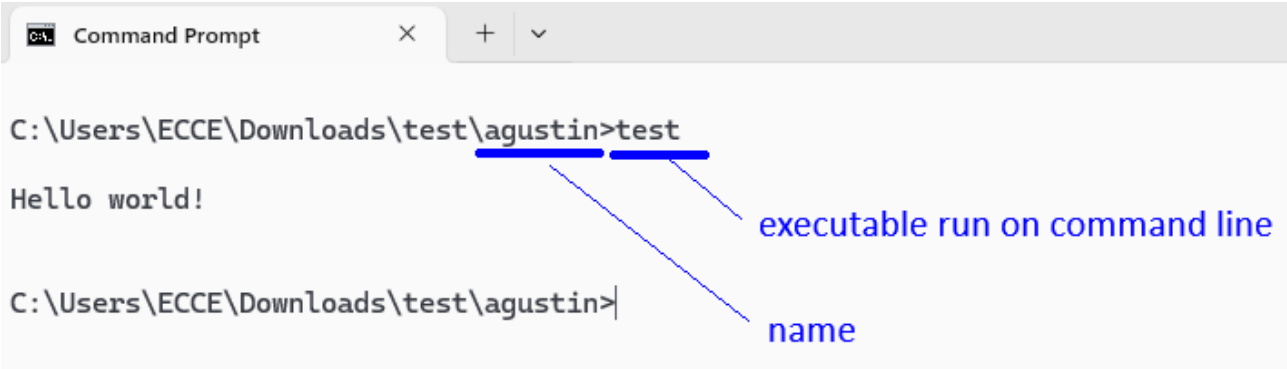
Special Cases

Score for the entire project is 0 if
* no source files are submitted
* the source files, as submitted, will not compile as a C++ project
* the implementation uses resources that are not allowed

early work

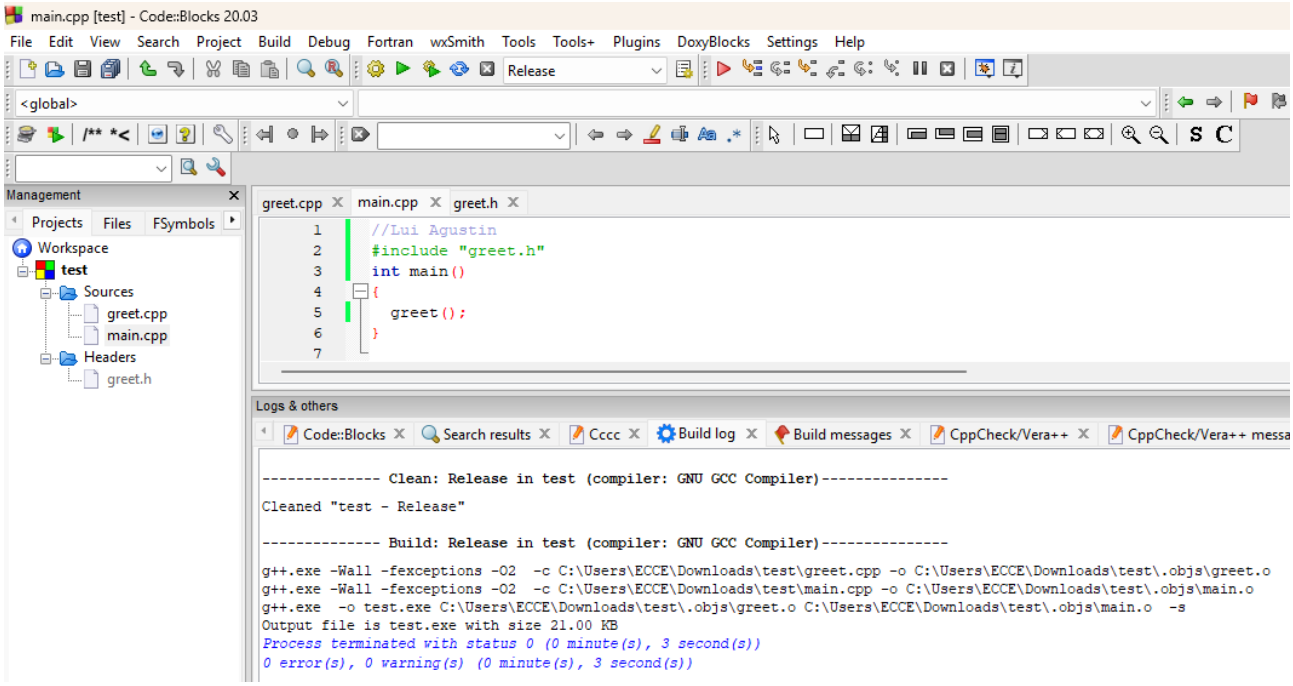
Early work credits are based on the final code submitted, assuming the final and early work code use the same style. The total number of lines in the final code that are not in the early work version, or that were modified from the early work version, other than formatting changes that do not modify compiled code, must not be more than 10% of the number of lines in the final code, excluding blank lines.

Implementation and Testing Environment (4 pts)
1 - all details of implementation platform are provided
1 - screenshot of compilation: all instructions are followed correctly
1 - screenshot of program run on the console: all instructions are followed correctly
1 - all filenames appearing in the screenshots are consistent with files in the project submission



sample screenshot - executable test.exe run by typing its name at the prompt from a path with name of author

ENGG 27.01 - Advanced Engineering Math with Numerical Methods, Lecture
ENGG 27.01 M 2025-2
Luisito L. Agustin

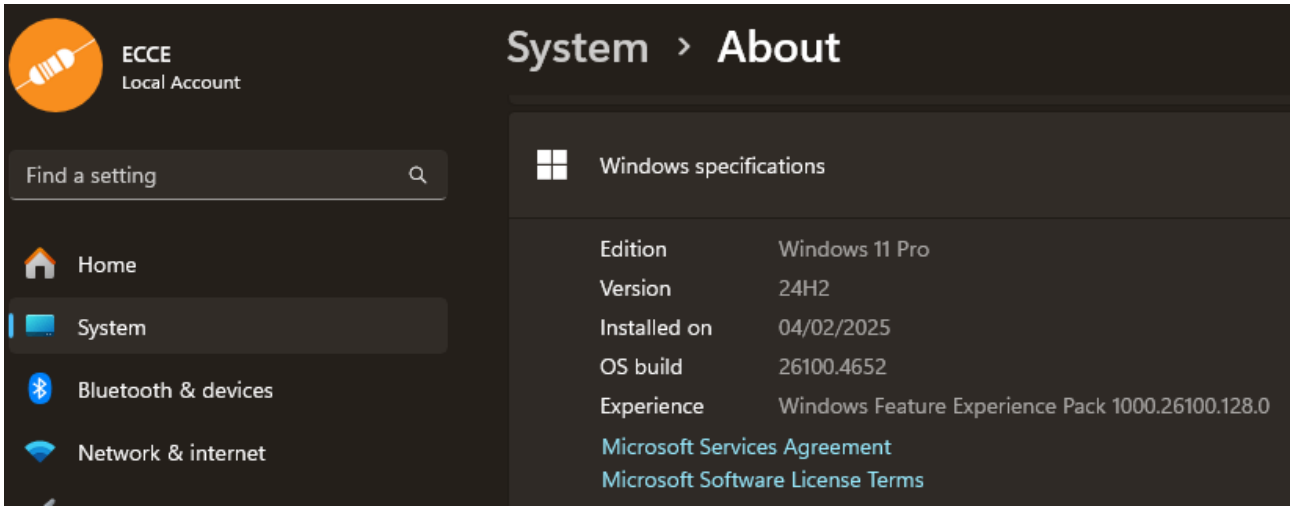


sample screenshot - compilation on Code::Blocks

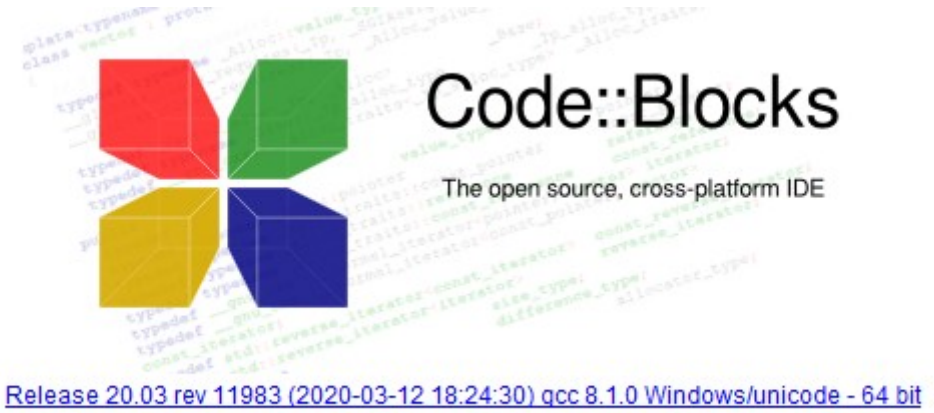
```
C:\Users\ECCE\codeblocks\MinGW\bin>g++ -v
Using built-in specs.
COLLECT_GCC=g++
COLLECT_LTO_WRAPPER=C:/Users/ECCE/codeblocks/MinGW/bin/../libexec/gcc/x86_64-w64-mingw32/8.1.0/lto-wrapper.exe
Target: x86_64-w64-mingw32
Configured with: ../../../src/gcc-8.1.0/configure --host=x86_64-w64-mingw32 --build=x86_64-w64-mingw32 --target=x86_64-w64-mingw32 --prefix=/mingw64
 --with-sysroot=/c/mingw810/x86_64-810-posix-seh-rt_v6-rev0/mingw64 --enable-shared --enable-static --disable-multilib --enable-languages=c,c++,fort
ran,lto --enable-libstdcxx-time=yes --enable-threads=posix --enable-libgomp --enable-libatomic --enable-lto --enable-graphite --enable-checking=rele
ase --enable-fully-dynamic-string --enable-version-specific-runtime-libs --disable-libstdcxx-pch --disable-libstdcxx-debug --enable-bootstrap --disa
ble-rpath --disable-win32-registry --disable-nls --disable-werror --disable-symvers --with-gnu-as --with-gnu-ld --with-arch=nocona --with-tune=core2
 --with-libiconv --with-system-zlib --with-gmp=/c/mingw810/prerequisites/x86_64-w64-mingw32-static --with-mpfr=/c/mingw810/prerequisites/x86_64-w64-
mingw32-static --with-mpc=/c/mingw810/prerequisites/x86_64-w64-mingw32-static --with-isl=/c/mingw810/prerequisites/x86_64-w64-mingw32-static --with-
pkgversion='x86_64-posix-seh-rev0, Built by MinGW-W64 project' --with-bugurl=https://sourceforge.net/projects/mingw-w64 CFLAGS='-O2 -pipe -fno-ident
 -I/c/mingw810/x86_64-810-posix-seh-rt_v6-rev0/mingw64/opt/include -I/c/mingw810/prerequisites/x86_64-zlib-static/include -I/c/mingw810/prerequisite
s/x86_64-w64-mingw32-static/include' CXXFLAGS='-O2 -pipe -fno-ident -I/c/mingw810/x86_64-810-posix-seh-rt_v6-rev0/mingw64/opt/include -I/c/mingw810/
prerequisites/x86_64-zlib-static/include -I/c/mingw810/prerequisites/x86_64-w64-mingw32-static/include' CPPFLAGS=' -I/c/mingw810/x86_64-810-posix-se
h-rt_v6-rev0/mingw64/opt/include -I/c/mingw810/prerequisites/x86_64-zlib-static/include -I/c/mingw810/prerequisites/x86_64-w64-mingw32-static/includ
e' LDFLAGS='-pipe -fno-ident -L/c/mingw810/x86_64-810-posix-seh-rt_v6-rev0/mingw64/opt/lib -L/c/mingw810/prerequisites/x86_64-zlib-static/lib -L/c/m
ingw810/prerequisites/x86_64-w64-mingw32-static/lib '
Thread model: posix
gcc version 8.1.0 (x86_64-posix-seh-rev0, Built by MinGW-W64 project)

C:\Users\ECCE\codeblocks\MinGW\bin>
```

compiler - gcc version 8.1.0 (x86_64-posix-seh-rev0, Built by MinGW-W64 project)



operating system - Windows 11 Pro, 24H2, build 26100.4652

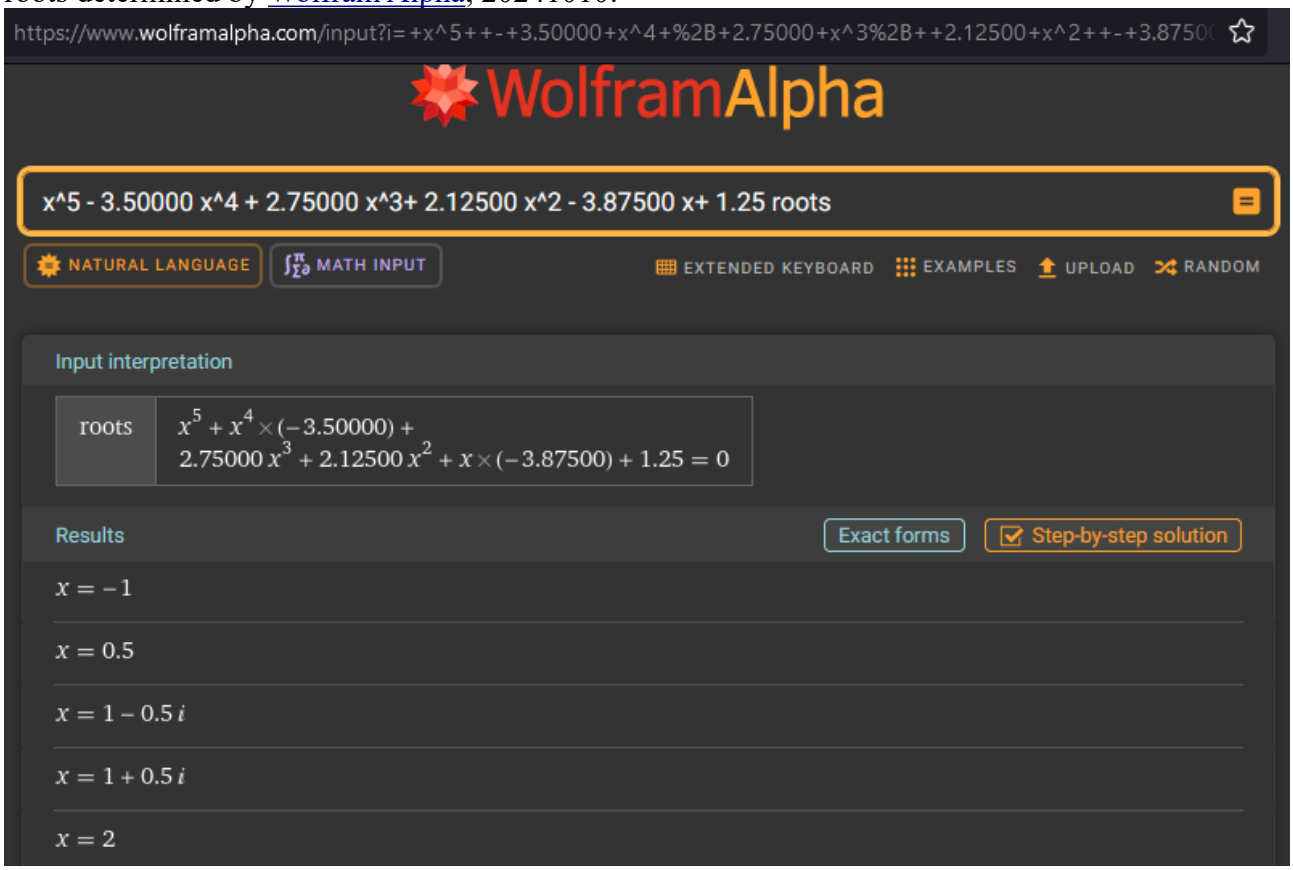


IDE -Code::Blocks 20.03
rev 11983 (2020-03-12 18:24:30) gcc 8.1.0 Windows/unicode - 64 bit

self-evaluation
0 - not done
2 - self-evaluation score differs from project score by more than 10 points or scores not tallied properly
4 - self-evaluation accurate (or evaluating this item leads to an error)

Notes

roots determined by Wolfram Alpha, 20241010:



Appendix: Polynomial File Format

See next page.

Polynomial File Format

Luisito L. Agustin
2024 10 09

This document specifies a text file format for representing polynomials with real-valued coefficients. The polynomials being represented are of the form

p(x) = a_0 + a_1 x + a_2 x^2 + . . . + a_{n-1} x^{n-1} + a_n x^n

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_{n-1} x^{n-1} + a_n x^n$$

of degree *n* in a single variable *x*. The variable name itself is not explicitly part of the specification. The file name extension is not part of the specification. Routines that process polynomial files must check the contents of the file regardless of the file name extension.

A polynomial is specified thru the first tokens extracted from consecutive lines of a text file, starting with the first line. These tokens may be preceded by any amount of white space and may be followed on the same line by additional text that may be treated as comments or otherwise ignored as far as specifying the polynomial is concerned.

POLYNOMIAL *optional comments*
n *optional comments*
a_0 *optional comments*
a_1 *optional comments*
.
.
.
a_{n-1} *optional comments*
a_n *optional comments*
optional comments

The first token on the first line must be the string “POLYNOMIAL” in uppercase. The first token on the second line must be a valid integer specifying the degree of the polynomial. Routines that process polynomials files must refer to this value in processing the rest of the file. The first tokens on subsequent lines must be valid floating point numbers representing the coefficients of the polynomial. The constant coefficient is specified first on line 3 followed by coefficients of increasing powers of the unspecified variable on subsequent lines.

To be considered compliant with the specifications, routines that process polynomial files must correctly identify polynomials from files that strictly comply with the specifications.

Compliant routines may accept polynomial specifications from files where the required string on the first line is not entirely in uppercase, or where it is absent. Compliant routines may also accept polynomial specifications where the required integer specifying the degree of the polynomial is absent, in which case the routine may instead determine the degree from the actual number of valid floating point coefficients extracted. The routine remains compliant provided that it also correctly identifies polynomials from files that strictly comply with the specifications.

Revision History - Polynomial File Format

2024 10 09	first version

Notes are not tracked as revisions to the specifications.

Notes

In C++, the extraction operator >> will skip white space in identifying tokens and will stop at white space.

The zero polynomial may be specified with a degree of 0 and a constant coefficient of 0 or 0.0, giving the zero polynomial the same status as other constant polynomials. However, there are references that consider the degree of the zero polynomial to be undefined or to have a value of -1. The specifications do not explicitly give any special status to the zero polynomial and hence, may not be adequate in applications where giving special status to the zero polynomial is critical.

ENGG 27.01 - Advanced Engineering Math with Numerical Methods, Lecture
ENGG 27.01 M 2025-2
Luisito L. Agustin

Roots of Polynomials

(Numerical Methods for Roots of Polynomials with Real-Valued Coefficients)

6.6 POLYNOMIALS

Polynomials are a special type of nonlinear algebraic equation of the general form

$$f_n(x) = a_1x^n + a_2x^{n-1} + \cdots + a_{n-1}x^2 + a_nx + a_{n+1} \tag{6.12}$$

where n is the order of the polynomial, and the a's are constant coefficients. In many (but not all) cases, the coefficients will be real. For such cases, the roots can be real and/or complex. In general, an nth order polynomial will have n roots.

Steven C. Chapra - *Applied Numerical Methods with MATLAB for Engineers and Scientists*, 3rd ed, McGraw-Hill, 2012

Geometry of multiple roots [edit]

A polynomial p can be written in the form of

$$p(x) = a(x - z_1)^{m_1} \cdots (x - z_k)^{m_k}$$

with distinct roots $z_1, \ldots, z_k$ and corresponding multiplicities $m_1, \ldots, m_k$. A root z_j is a **simple root** if $m_j = 1$ or a **multiple root** if $m_j \geq 2$. Simple roots are Lipschitz continuous with respect to coefficients but multiple roots are not. In other words, simple roots have bounded sensitivities but multiple roots are infinitely sensitive if the coefficients are perturbed arbitrarily. As a result, most root-finding algorithms suffer substantial loss of accuracy on multiple roots in numerical computation.

https://en.wikipedia.org/wiki/Geometrical_properties_of_polynomial_roots, accessed 20260310

Product of Roots of Polynomial

Theorem

Let P be the polynomial equation:

$$a_n z^n + a_{n-1} z^{n-1} + \cdots + a_1 z + a_0 = 0$$

such that $a_n \neq 0$.

The product of the roots of P is $\dfrac{(-1)^n a_0}{a_n}$.

https://proofwiki.org/wiki/Product_of_Roots_of_Polynomial, accessed 20260310

Proof

Let the roots of P be $z_1, z_2, \ldots, z_n$.

Then P can be written in factored form as:

$$a_n \prod_{k=1}^{n} (z - z_k) = a_0 (z - z_1)(z - z_2) \cdots (z - z_n)$$

Multiplying this out, P can be expressed as:

$$a_n \left(z^n - (z_1 + z_2 + \cdots + z_n) z^{n-1} + \cdots + (-1)^n z_1 z_2 \cdots z_n\right) = 0$$

where the coefficients of $z^{n-2}, z^{n-3}, \ldots$ are more complicated and irrelevant.

Equating powers of z, it follows that:

$$a_n (-1)^n z_1 z_2 \cdots z_n = a_0$$

from which:

$$z_1 z_2 \cdots z_n = \frac{(-1)^n a_0}{a_n}$$

■

https://proofwiki.org/wiki/Product_of_Roots_of_Polynomial, accessed 20260310

Complex conjugate root theorem

6 languages

Article Talk Tools

From Wikipedia, the free encyclopedia

In mathematics, the **complex conjugate root theorem** states that if P is a polynomial in one variable with real coefficients, and $a + bi$ is a root of P with a and b being real numbers, then its complex conjugate $a - bi$ is also a root of P.[1]

It follows from this (and the fundamental theorem of algebra) that, if the degree of a real polynomial is odd, it must have at least one real root.[2] That fact can also be proved by using the intermediate value theorem.

https://en.wikipedia.org/wiki/Complex_conjugate_root_theorem, accessed 20260310

One proof of the theorem is as follows:[2]

Consider the polynomial

$$P(z) = a_0 + a_1 z + a_2 z^2 + \cdots + a_n z^n$$

where all a_r are real. Suppose some complex number ζ is a root of P, that is $P(\zeta) = 0$. It needs to be shown that

$$P(\bar{\zeta}) = 0$$

as well.

If $P(\zeta) = 0$, then

$$a_0 + a_1 \zeta + a_2 \zeta^2 + \cdots + a_n \zeta^n = 0$$

which can be put as

$$\sum_{r=0}^{n} a_r \zeta^r = 0.$$

https://en.wikipedia.org/wiki/Complex_conjugate_root_theorem, accessed 20260310

Now

$$P(\overline{\zeta}) = \sum_{r=0}^{n} a_r (\overline{\zeta})^r$$

and given the properties of complex conjugation,

$$\sum_{r=0}^{n} a_r (\overline{\zeta})^r = \sum_{r=0}^{n} a_r \overline{\zeta^r} = \sum_{r=0}^{n} \overline{a_r \zeta^r} = \overline{\sum_{r=0}^{n} a_r \zeta^r}.$$

https://en.wikipedia.org/wiki/Complex_conjugate_root_theorem, accessed 20260310

Since

$$\overline{\sum_{r=0}^{n} a_r \zeta^r} = \overline{0},$$

it follows that

$$\sum_{r=0}^{n} a_r \left(\overline{\zeta}\right)^r = \overline{0} = 0.$$

That is,

$$P\left(\overline{\zeta}\right) = a_0 + a_1\overline{\zeta} + a_2\left(\overline{\zeta}\right)^2 + \cdots + a_n\left(\overline{\zeta}\right)^n = 0.$$

Note that this works only because the a_r are real, that is, $\overline{a_r} = a_r$. If any of the coefficients were non-real, the roots would not necessarily come in conjugate pairs. In addition, one can show that for any $\zeta \in \mathbb{C}$, it holds that $P(\overline{\zeta}) = \overline{P(\zeta)}$ even if $P(\zeta) \neq 0$.

https://en.wikipedia.org/wiki/Complex_conjugate_root_theorem, accessed 20260310

polynomials

* with real-valued coefficients

Easy Cases

* lines

* quadratic

* cubic

* real-valued roots

In mathematics, the **Abel–Ruffini theorem** (also known as **Abel's impossibility theorem**) states that there is no solution in radicals to general polynomial equations of degree five or higher with arbitrary coefficients. Here, *general* means that the coefficients of the equation are viewed and manipulated as indeterminates.

The theorem is named after Paolo Ruffini, who made an incomplete proof in 1799[1] (which was refined and completed in 1813[2] and accepted by Cauchy) and Niels Henrik Abel, who provided a proof in 1824.[3][4]

The term can also refer to the slightly stronger result that there are equations of degree five and higher that cannot be solved by radicals. This does not follow from Abel's statement of the theorem, but is a corollary of his proof, as his proof is based on the fact that some polynomials in the coefficients of the equation are not the zero polynomial. This improved statement follows directly from Galois theory § A non-solvable quintic example. Galois theory implies also that

$$x^5 - x - 1 = 0$$

is the simplest equation that cannot be solved in radicals, and that *almost all* polynomials of degree five or higher cannot be solved in radicals.

The impossibility of solving in degree five or higher contrasts with the case of lower degree: one has the quadratic formula, the cubic formula, and the quartic formula for degrees two, three, and four, respectively.

https://en.wikipedia.org/wiki/Abel%E2%80%93Ruffini_theorem, accessed 20260310

Survey of Methods

Finding one root [edit]

The most widely used method for computing a root of any differentiable function f is Newton's method, in which an initial guess x_0 is iteratively refined. At each iteration the tangent line to f at x_n is used as a linear approximation to f, and its root is used as the succeeding guess x_{n+1}:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)},$$

In general, the value of x_n will converge to a root of f.

In particular, the method can be applied to compute a root of polynomial functions. In this case, the computations in Newton's method can be accelerated using Horner's method or evaluation with preprocessing for computing the polynomial and its derivative in each iteration.

Though the rate of convergence of Newton's method is generally quadratic, it might converge much slowly or even not converge at all. In particular, if the polynomial has no real root, and x_0 is chosen to be a real number, then Newton's method cannot converge. However, if the polynomial has a real root, which is larger than the larger real root of its derivative, then Newton's method converges quadratically to this largest root if x_0 is larger than this larger root (there are easy ways for computing an upper bound of the roots, see Properties of polynomial roots). This is the starting point of Horner's method for computing the roots.

https://en.wikipedia.org/wiki/Polynomial_root-finding, accessed 20260313

Finding complex roots in pairs [edit]

If the given polynomial only has real coefficients, one may wish to avoid computations with complex numbers. To that effect, one has to find quadratic factors for pairs of conjugate complex roots. The application of the multidimensional Newton's method to this task results in Bairstow's method.

The real variant of Jenkins–Traub algorithm is an improvement of this method.

https://en.wikipedia.org/wiki/Polynomial_root-finding, accessed 20260313

Finding all complex roots [edit]

Methods using complex-number arithmetic [edit]

Both the Aberth method and the similar yet simpler Durand–Kerner method simultaneously find all of the roots using only simple complex number arithmetic. The Aberth method is presently the most efficient method. Accelerated algorithms for multi-point evaluation and interpolation similar to the fast Fourier transform can help speed them up for large degrees of the polynomial.

A free implementation of Aberth's method is available under the name of MPSolve. This is a reference implementation, which can find routinely the roots of polynomials of degree larger than 1,000, with more than 1,000 significant decimal digits.

Another method with this style is the Dandelin–Gräffe method (sometimes also ascribed to Lobachevsky), which uses polynomial transformations to repeatedly and implicitly square the roots. This greatly magnifies variances in the roots. Applying Viète's formulas, one obtains easy approximations for the modulus of the roots, and with some more effort, for the roots themselves.

https://en.wikipedia.org/wiki/Polynomial_root-finding, accessed 20260313

Fixed-Point Iteration

6.1 SIMPLE FIXED-POINT ITERATION

As just mentioned, open methods employ a formula to predict the root. Such a formula can be developed for simple *fixed-point iteration* (or, as it is also called, *one-point iteration* or *successive substitution*) by rearranging the function $f(x) = 0$ so that x is on the left-hand side of the equation:

$$x = g(x) \tag{6.1}$$

This transformation can be accomplished either by algebraic manipulation or by simply adding x to both sides of the original equation.

The utility of Eq. (6.1) is that it provides a formula to predict a new value of x as a function of an old value of x. Thus, given an initial guess at the root x_i, Eq. (6.1) can be used to compute a new estimate x_{i+1} as expressed by the iterative formula

$$x_{i+1} = g(x_i) \tag{6.2}$$

Steven C. Chapra - *Applied Numerical Methods with MATLAB for Engineers and Scientists*, 3rd ed, McGraw-Hill, 2012

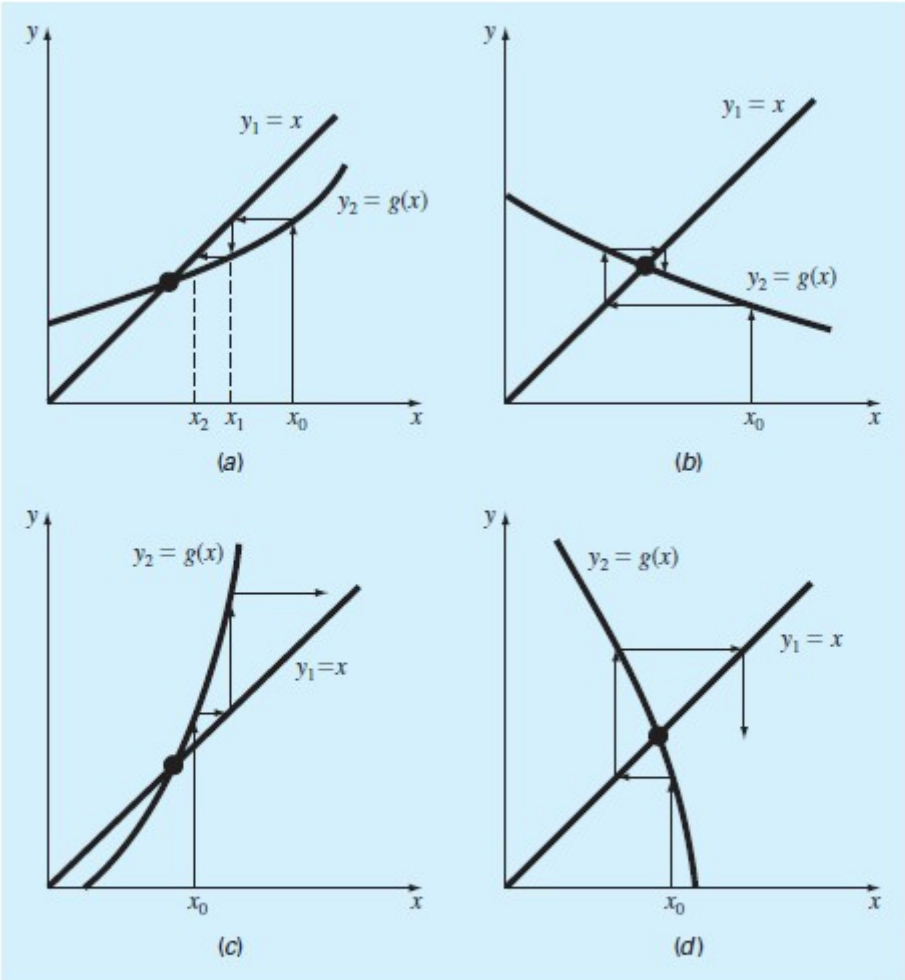


FIGURE 6.3
Graphical depiction of (*a*) and (*b*) convergence and (*c*) and (*d*) divergence of simple fixed-point iteration. Graphs (*a*) and (*c*) are called monotone patterns whereas (*b*) and (*c*) are called oscillating or spiral patterns. Note that convergence occurs when $|g'(x)| < 1$.

Steven C. Chapra - *Applied Numerical Methods with MATLAB for Engineers and Scientists*, 3rd ed, McGraw-Hill, 2012

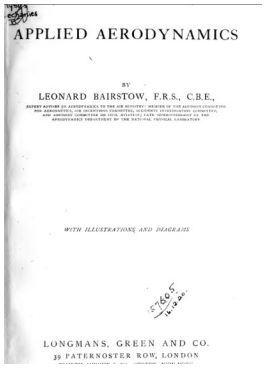
Bairstow's method

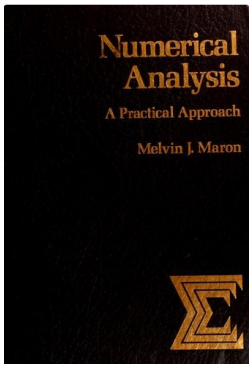
References

(online article) Wikipedia: *Bairstow's method*, https://en.wikipedia.org/wiki/Bairstow%27s_method, as of 20240925.

(book) Leonard Bairstow: *Applied Aerodynamics*, Longmans, Green And Co., 1920, Internet Archive: https://archive.org/details/appliedaerodynam00bairuoft, as of 20240925. (Bairstow's Method is in the appendix: *The solution of algebraic equations with numerical coefficients in the case where several pairs of complex roots exist*)

(book) Melvin Maron: *Numerical Analysis, A Practical Approach*, 1982 (available thru Open Library, https://openlibrary.org/books/OL4262608M/Numerical_analysis as of 20240927).





2.5 Finding Roots of Real Polynomials; Bairstow's Method

The stability of electrical or mechanical systems is related to the real part of the *complex* roots of certain polynomials whose degree can be as high as 30. It has been proved that general formulas such as the quadratic formula *cannot* be found for the roots of polynomials of degree $\geqslant 5$. So we generally have no choice but to use a numerical method to find such roots.

By changing the REAL variables to COMPLEX (and suitably modifying the input/output statements) in a FORTRAN program for the Newton–Raphson method, one can make it find complex roots. However, this approach is quite cumbersome if automated complex arithmetic is not available (e.g., for a hand computation). We now present a more desirable procedure.

Algorithm: Quadratic Deflation

Purpose: To find all roots of a real polynomial $p(x) = a_1x^n + \cdots + a_nx + a_{n+1}$

{*initialize*}
GET $n, a_1, \ldots, a_{n+1}$ $\{a_1 \neq 0\}$

{*iterate*}
DO WHILE $n \geqslant 2$
  BEGIN
  {**factor**} Find a real quadratic $q(x)$ such that $p(x) = q(x)Q(x)$
  {**solve**} Find the two roots r_1, r_2 of $q(x)$ by the quadratic formula
  OUTPUT (Roots: r_1, r_2)
  {**deflate**} BEGIN $p(x) \leftarrow Q(x)$; $n \leftarrow n - 2$ END
  END

IF $n = 1$ THEN BEGIN *LastRoot* $\leftarrow -a_2/a_1$; OUTPUT (Root: *LastRoot*) END

FIGURE 2.5-1 PSEUDOPROGRAM FOR QUADRATIC DEFLATION.

2.5A DEFLATING REAL POLYNOMIALS USING QUADRATIC FACTORS

Even if available, complex arithmetic should not be necessary when $p(x)$ is a **real polynomial** [i.e., when all coefficients of $p(x)$ are real]. This is because *the complex roots of real polynomials occur in conjugate pairs.* For any such pair, say $\alpha \pm i\beta$, the real quadratic

$$q(x) = (x - \alpha - i\beta)(x - \alpha + i\beta) = (x - \alpha)^2 + \beta^2$$

is a *real* factor of $p(x)$, that is, there is a *real* polynomial $Q(x)$ such that

$$p(x) = q(x)Q(x), \qquad \text{where degree } Q(x) = \text{degree } p(x) - 2$$

This suggests that when $n \geqslant 3$, all roots of the *real* polynomial $p(x)$ can be found using the **Quadratic Deflation** algorithm described in Figure 2.5-1.

The **solve** step of the Quadratic Deflation algorithm can be performed by a call to a subroutine such as QROOTS in Figure 1.5-4. The method we will use to find a quadratic factor in the **factor** step is due to Bairstow. It uses real arithmetic and converges quadratically when the roots of $q(x)$ are *not* also roots of $Q(x) = p(x)/q(x)$. The basis for Bairstow's method is the synthetic division algorithm described next.

2.5B SYNTHETIC DIVISION BY $q(x)$

If the real nth-degree polynomial

$$p(x) = a_1x^n + a_2x^{n-1} + \cdots + a_nx + a_{n+1} \qquad (a_1 \neq 0) \tag{1}$$

is divided by a real quadratic

$$q(x) = x^2 - rx - s \qquad (\textit{notice the minus signs!}) \tag{2}$$

the result will be a *quotient* $Q(x)$ of degree $n - 2$ and a *remainder* $R(x)$ of degree $\leqslant 1$. This division can be displayed as

$$\frac{p(x)}{q(x)} = Q(x) + \frac{R(x)}{q(x)} \qquad \text{or} \qquad p(x) = q(x)Q(x) + R(x) \tag{3}$$

Our notation for this division will be as follows (notice the b_n term):

$$p(x) = \underbrace{(x^2 - rx - s)}_{q(x)}\,\underbrace{(b_1x^{n-2} + b_2x^{n-3} + \cdots + b_{n-2}x + b_{n-1})}_{Q(x)} + \underbrace{b_n(x - r) + b_{n+1}}_{R(x)} \tag{4}$$

Expanding (4) and equating coefficients with (1) gives the following table.

Power of x	Equated Coefficients	Solving for b_i
x^n	$a_1 = b_1$	$b_1 = a_1$
x^{n-1}	$a_2 = b_2 - rb_1$	$b_2 = a_2 + rb_1$
x^{n-2}	$a_3 = b_3 - rb_2 - sb_1$	$b_3 = a_3 + rb_2 + sb_1$
x^{n-3}	$a_4 = b_4 - rb_3 - sb_2$	$b_4 = a_4 + rb_3 + sb_2$
$\vdots$	$\vdots$	$\vdots$
x	$a_n = b_n - rb_{n-1} - sb_{n-2}$	$b_n = a_n + rb_{n-1} + sb_{n-2}$
constant	$a_{n+1} = b_{n+1} - rb_n - sb_{n-1}$	$b_{n+1} = a_{n+1} + rb_n + sb_{n-1}$

(5)

The right-hand column shows that the b_i's that define $Q(x)$ and $R(x)$ can be obtained **synthetically** [i.e., using only r, s and the coefficients of $p(x)$, as shown in Figure 2.5-2(a)].

(a)

	a_1	a_2	a_3	a_4	$\cdots$	a_{n-1}	a_n	a_{n+1}
$\times r>$		$+b_1r$	$+b_2r$	$+b_3r$	$\cdots$	$+b_{n-2}r$	$+b_{n-1}r$	$+b_nr$
$\times s>$			$+b_1s$	$+b_2s$	$\cdots$	$+b_{n-3}s$	$+b_{n-2}s$	$+b_{n-1}s$
	b_1	b_2	b_3	b_4	$\cdots$	b_{n-1}	b_n	b_{n+1}

This quadratic **synthetic division** algorithm is illustrated in Figure 2.5-2(b) for the division

$$\frac{2x^6 - 4x^5 + x^3 - 40x}{x^2 - x + 4} = (2x^4 - 2x^3 - 10x^2 - x + 39) + \frac{3(x-1) - 153}{x^2 - x + 4} \tag{6}$$

(b) a_i's>	2	−4	0	1	0	−40	0
×1>		2	−2	−10	−1	39	3
×(−4)>			−8	8	40	4	−156
b_i's>	2	−2	−10	−1	+39	+3	−153

FIGURE 2.5-2 QUADRATIC SYNTHETIC DIVISION: (a) $p(x)/(x^2 - rx - s)$; (b) $(2x^6 - 4x^5 + x^3 - 40x)/(x^2 - x + 4)$.

2.5C BAIRSTOW'S METHOD

Our objective is to find quadratics $q(x)$ which are factors of $p(x)$, that is, which make $R(x) = b_n(x - r) + b_{n+1} = 0$. To do this, we must find r and s that make

$$b_n = 0 \qquad \text{and} \qquad b_{n+1} = 0 \tag{7}$$

This is not a trivial problem because, as can be seen from (5), each of $b_2, \ldots, b_{n+1}$ depends upon the preceding b_i's, which are themselves functions of r and s.

Let $\bar{r}$ and $\bar{s}$ be values that satisfy (7), and suppose that r and s are approximations of $\bar{r}$ and $\bar{s}$. If the increments

$$dr = \bar{r} - r \qquad \text{and} \qquad ds = \bar{s} - s \tag{8}$$

are small and we view b_n and b_{n+1} as functions of (r, s), then we can use the total differentials of b_n and b_{n+1} to get the approximations

$$0 = b_n(\bar{r}, \bar{s}) = b_n(r + dr, s + ds) \approx b_n(r, s) + \frac{\partial b_n}{\partial r}\, dr + \frac{\partial b_n}{\partial s}\, ds \tag{9a}$$

$$0 = b_{n+1}(\bar{r}, \bar{s}) = b_{n+1}(r + dr, s + ds) \approx b_{n+1}(r, s) + \frac{\partial b_{n+1}}{\partial r}\, dr + \frac{\partial b_{n+1}}{\partial s}\, ds \tag{9b}$$

where the partial derivatives are evaluated at (r, s) [see Appendix II-5C]. Hence, if r and s are kth iterates (i.e., $r = r_k$ and $s = s_k$) and we use dr_k and ds_k to denote the solution of (9) with "$\approx$" replaced by "$=$", then in view of (8), r_{k+1} and s_{k+1} defined by

$$r_{k+1} = r_k + dr_k \qquad \text{and} \qquad s_{k+1} = s_k + ds_k \tag{10}$$

should approximate $\bar{r}$ and $\bar{s}$ better than r_k and s_k.

In order to be able to use iteration (10), we must know the values of the four partial derivatives in (9). What Bairstow astutely observed is that if synthetic division by

$$q(x) = x^2 - rx - s$$

is performed *with the b_i's replacing the a_i's* to get $c_1, \ldots, c_n$ as follows:

$$\begin{array}{c|cccccc}
 & b_1 & b_2 & b_3 & \cdots & b_n & b_{n+1} \\
\hline
\times r > & & +c_1 r & +c_2 r & & +c_{n-1} r & \\
\times s > & & & +c_1 s & & +c_{n-2} s & \\
\hline
 & c_1 & c_2 & c_3 & \cdots & c_n & \underbrace{\qquad}_{\text{not needed}}
\end{array} \tag{11}$$

then the required partial derivatives in (9) can be obtained as

$$\frac{\partial b_n}{\partial r} = c_{n-1}, \quad \frac{\partial b_n}{\partial s} = c_{n-2}, \quad \frac{\partial b_{n+1}}{\partial r} = c_n, \quad \text{and} \quad \frac{\partial b_{n+1}}{\partial s} = c_{n-1} \tag{12}$$

A proof of (12) will be given in Section 2.5E. From (12) and (9) we see that dr_k and ds_k in (10) are obtained by solving the *linear* equations

$$\begin{aligned} c_{n-1}\,dr + c_{n-2}\,ds &= -b_n \\ c_n\,dr + c_{n-1}\,ds &= -b_{n+1} \end{aligned} \tag{13}$$

Using Cramer's rule [see (12) of Section 3.5D], we get

$$dr_k = \frac{b_n c_{n-1} - b_{n+1} c_{n-2}}{c_n c_{n-2} - c_{n-1}^2} \quad \text{and} \quad ds_k = \frac{b_{n+1} c_{n-1} - b_n c_n}{c_n c_{n-2} - c_{n-1}^2} \tag{14}$$

For large roots, try $r_0 = -\dfrac{a_2}{a_1}$ and $s_0 = -\dfrac{a_3}{a_1}$ **(15a)**

For small roots (if $a_{n-1} \neq 0$), try $r_0 = -\dfrac{a_n}{a_{n-1}}$ and $s_0 = -\dfrac{a_{n+1}}{a_{n-1}}$ **(15b)**

This is because very large roots $\bar{x}$ will satisfy

$$0 = p(\bar{x}) \approx (a_1\bar{x}^2 + a_2\bar{x} + a_3)\,\bar{x}^{n-2} \qquad \text{so that} \qquad \bar{x}^2 + \frac{a_2}{a_1}\bar{x} + \frac{a_3}{a_1} \approx 0$$

and, similarly, very small roots $\bar{x}$ satisfy $a_{n-1}\bar{x}^2 + a_n\bar{x} + a_{n+1} \approx 0$.

EXAMPLE. Find all roots of the fifth-degree polynomial

$$p(x) = x^5 - 4.5x^4 + 4.55x^3 + 2.675x^2 - 3.3x - 1.4375 \tag{16}$$

using Bairstow's method. Try to find $\bar{x}$'s of smallest magnitude first.

Solution. Using (15b), we try as initial guesses for r and s,

$$r_0 = -\frac{-3.3}{2.675} \doteq 1.233645 \qquad \text{and} \qquad s_0 = -\frac{-1.4375}{2.675} \doteq 0.5373832$$

The c_i values required to get dr_0 and ds_0 are obtained using the "double" synthetic division as follows:

a_i's:	1	−4.5	+4.55	+2.675	−3.3	−1.4375
×1.233645>		+1.233645	−4.029522	+1.305025	+2.744540	+0.016057
×0.5373832>			+0.537383	−1.755284	+0.568477	+1.195538
b_i's:	1	−3.266355	+1.057861	+2.224740	+0.013016	−0.225904
×1.233645>		+1.233645	−2.507643	−1.125576	+0.008414	
×0.5373832>			+0.537383	−1.092344	−0.490308	
c_i's:	1	−2.032710	−0.912398	+0.006820	−0.468877	

The desired increments are then obtained using (14):

$$dr_0 = \frac{(0.013016)(0.006820) - (-0.225904)(-0.912398)}{(-0.468877)(-0.912398) - (0.006820)^2} = \frac{-0.2060261}{0.4277564} = -0.4816435$$

$$ds_0 = \frac{(0.006820)(-0.225904) - (-0.468877)(0.013016)}{-0.4277564} = +0.01066556$$

Adding these increments to the initial values, we get

$$r_1 = r_0 + dr_0 = 1.233645 - 0.4816435 = 0.752001$$
$$s_1 = s_0 + ds_0 = 0.5373832 + 0.01066556 = 0.5480488$$

If we continue the iteration, we get Table 2.5-1.

TABLE 2.5-1 CONVERGENCE OF BAIRSTOW'S METHOD.

k	Δr	Δs	r	s
0	—	—	1.233645	0.5373832
1	−0.4816435	0.0106656	0.752001	0.5480488
2	0.1478756	0.2232268	0.899877	0.7712756
3	0.5234931	0.1512589	1.423370	0.9225345
4	−0.0828045	−0.7647468	1.340565	0.1577877
5	0.6070840	0.3324581	1.948274	0.4902458
6	−2.8041133	−0.6506271	−0.855840	−0.1603809
7	−0.1514431	−0.0790732	−1.007283	−0.2394512
8	0.766207E−2	−0.990590E−2	0.999620	−0.2493600
9	0.379778E−3	−0.640298E−3	−1.000000	−0.2500003
10	0.262E−6	0.3218E−6	−1.000000	−0.2500000

We see from the Δr and Δs columns of Table 2.5-1 that *the convergence was quadratic once* r_k *and* s_k *were near* $\bar{r} = -1$ *and* $\bar{s} = -0.25$ (i.e., for $k \geqslant 7$). However, *both* r_k *and* s_k *"wandered" for the first six iterations.* As it turned out, the roots of the quadratic factor obtained, namely

$$q(x) = x^2 - (-1)x - (-0.25) = (x + 0.5)^2 \qquad (-0.5 \text{ is a double root}) \qquad \textbf{(17)}$$

are the two roots of $p(x)$ of smallest magnitude [see (21)].

The initial "wandering" can go on for a long time before converging. In fact, had $r_0 = s_0 = 0$ been used as initial guesses, then over 30 iterations (for which some of the increments are larger than 1000!) would have been needed before finally finding the $q(x)$ in (17). However, it could just as well have found a different quadratic factor or not converged at all. In view of the amount of work needed for a single iteration, *this method is not recommended for hand calculation unless good initial guesses are known.*

Dividing $p(x)$ synthetically by $q(x) = x^2 + x + 0.25$, we get

a_i's >	1	−4.5	+4.55	+2.675	−3.3	−1.4375
×(−1) >		−1.0	+5.5	−9.8	+5.75	0
×(−0.25) >			−0.25	+1.375	−2.45	+1.4375
b_i's >	1	−5.5	+9.8	−5.75	0	0

so that

$$p(x) = (x + 0.5)^2\, Q(x), \qquad \text{where } Q(x) = x^3 - 5.5x^2 + 9.8x - 5.75 \tag{18}$$

By (15b), initial guesses for getting the smallest roots of $Q(x)$ are

$$r_0 = -\frac{9.8}{-5.5} \doteq 1.7818182 \qquad \text{and} \qquad s_0 = -\frac{-5.75}{-5.5} \doteq -1.0454545 \tag{19}$$

The iterations of Bairstow's method, with $Q(x)$ in (18) taken as $p(x)$ and r_0 and s_0 given in (19), are shown in Table 2.5-2. After only eight iterations, Bairstow's method found that

$$q(x) = x^2 - (3)x - (-2.3) = (x - 1.5)^2 + 0.05 \tag{20}$$

This irreducible quadratic has a pair of complex conjugate roots $\bar{x} = 1.5 \pm \sqrt{0.05}\,i$. Dividing the deflated $p(x)$ [i.e., $Q(x)$ in (18)] by this $q(x)$ leaves $x - 2.5$; so the five roots of $p(x)$ in (16) are

$$-0.5, \quad -0.5, \quad 1.5 + \sqrt{0.05}\,i, \quad 1.5 - \sqrt{0.05}\,i, \quad 2.5 \tag{21}$$

In fact, the graph of $p(x)$ was given in Figure 2.3-1.

TABLE 2.5-2 BAIRSTOW'S METHOD FOR THE DEFLATED $p(x)$.

k	Δr	Δs	r	s
0	—	—	1.718182	−1.045455
1	3.500277	8.145488	2.454822	9.927306
2	−0.0652019	−12.29043	2.389620	−2.363122
3	0.3506920	0.2485134	2.740312	−2.114609
4	0.2078440	−0.1189577	2.948156	−2.233567
5	0.508473E−1	−0.633505E−1	2.999003	−2.296917
6	0.999638E−3	−0.308328E−2	3.000003	−2.300001
7	−0.30177E−5	0.5096E−6	3.000000	−2.300000

2.5D HOW INACCURATE COEFFICIENTS CAN AFFECT THE LOCATION OF ROOTS

The reader should be aware of the fact that the problem of finding roots of a polynomial $p(x)$ is **ill conditioned** in the sense that *small changes in the coefficients of $p(x)$ can cause disproportionately large changes in the location of its roots.* The ill conditioning generally becomes more severe as the degree of $p(x)$ and the magnitude of its coefficients increase. For example, the sixth-degree polynomial

$$p(x) = x^6 - 21x^5 + 175x^4 - 735x^3 + 1624x^2 - 1764x + 720 \tag{22}$$

has six simple roots: $\bar{x} = 1, 2, 3, 4, 5$, and 6. Table 2.5-3 shows how these roots move as a_5, the coefficient of x^2, is changed from 1624.0 to 1624.25 to 1624.5 without changing the other coefficients. Increasing the coefficient of x^2 by $\frac{1}{4}$ in the fifth significant digit caused changes in the *second* significant digit of $\bar{x}_3$, $\bar{x}_4$, $\bar{x}_5$, and $\bar{x}_6$. And adding another $\frac{1}{4}$ (a net change of about 0.03%) caused $\bar{x}_3$ and $\bar{x}_4$ to become *complex*! If a_5 is further increased to 1625 (a net change of about 0.06%), then $\bar{x}_5$ and $\bar{x}_6$ become complex as well.

The coefficients in (22) are all integers and hence can be stored without roundoff error. However, when the coefficients *cannot* be stored exactly, inherent roundoff alone may result in a stored polynomial whose roots do not even resemble those of the intended polynomial. Consequently, *extended precision is advised when seeking roots of polynomials of large degree* (say $n \geqslant 5$), especially if some of the coefficients are large. Also, any roots found after deflating should be used as initial guesses for roots of the *original* $p(x)$ to eliminate the effects of propagated roundoff in deflating.

TABLE 2.5-3 SENSITIVITY OF SIX ROOTS OF $p(x)$ TO SMALL CHANGES IN a_5.

a_5	$\bar{x}_1$	$\bar{x}_2$	$\bar{x}_3$	$\bar{x}_4$	$\bar{x}_5$	$\bar{x}_6$
1624.00	1.000000	2.000000	3.000000	4.000000	5.000000	6.000000
1624.25	1.002102	1.961491	3.268076	3.621456	5.236785	5.910090
1624.50	1.004243	1.927929	$(3.407987 \pm 0.3728825i)$		5.251515	5.730340

2.5E PROOF OF THE BAIRSTOW'S METHOD FORMULAS

We wish to show by induction that if the b_j's are obtained synthetically from $a_1, \ldots, a_{n+1}$, that is, if

$$b_1 = a_1; \quad b_2 = a_2 + b_1 r; \quad b_j = a_j + b_{j-1} r + b_{j-2} s \qquad \text{for } j = 3, \ldots, n+1 \tag{23a}$$

and the c_j's are obtained synthetically from $b_1, \ldots, b_n$, that is, if

$$c_1 = b_1; \quad c_2 = b_2 + c_1 r; \quad c_j = b_j + c_{j-1} r + c_{j-2} s \qquad \text{for } j = 3, \ldots, n \tag{23b}$$

then the partial derivatives of b_i satisfy

$$\frac{\partial b_i}{\partial r} = c_{i-1} \quad \text{and} \quad \frac{\partial b_i}{\partial s} = c_{i-2} \qquad \text{for } i = 3, 4, \ldots, n \tag{24}$$

When $i = 3$ (to begin the induction) we first note from (23a) that

$$\frac{\partial b_1}{\partial r} = \frac{\partial b_1}{\partial s} = 0, \quad \text{so} \frac{\partial b_2}{\partial r} = 0 + \left[b_1 \cdot 1 + \frac{\partial b_1}{\partial r} r \right] = b_1 \quad \text{and} \quad \frac{\partial b_2}{\partial s} = 0 + \frac{\partial b_1}{\partial s} r = 0 \tag{25}$$

(Recall that $\partial/\partial r$ is with s fixed, and $\partial/\partial s$ is with r fixed.) Hence

$$\frac{\partial b_3}{\partial r} = 0 + \left[b_2 \cdot 1 + \frac{\partial b_2}{\partial r} r \right] + \frac{\partial b_1}{\partial r} s = b_2 + b_1 r = b_2 + c_1 r = c_2$$

$$\frac{\partial b_3}{\partial s} = 0 + \frac{\partial b_2}{\partial s} r + \left[\frac{\partial b_1}{\partial s} s + b_1 \cdot 1 \right] = b_1 = c_1$$

by (25) and (23). Similarly, assuming that (24) holds for $j = 3, \ldots, i-1$, we get

$$\frac{\partial b_i}{\partial r} = 0 + \left[b_{i-1} \cdot 1 + \frac{\partial b_{i-1}}{\partial r} r \right] + \frac{\partial b_{i-2}}{\partial r} s = b_{i-1} + c_{i-2} r + c_{i-3} s = c_{i-1}$$

$$\frac{\partial b_i}{\partial s} = 0 + \frac{\partial b_{i-1}}{\partial s} r + \left[\frac{\partial b_{i-2}}{\partial s} s + b_{i-2} \cdot 1 \right] = b_{i-2} + c_{i-3} r + c_{i-4} s = c_{i-2}$$

This completes the induction and hence the proof of (24).

Durand–Kerner Method

Durand–Kerner method

3 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

This article **has an unclear citation style**. The references used may be made clearer with a different or consistent style of citation and footnoting. *(November 2020) (Learn how and when to remove this message)*

In numerical analysis, the **Weierstrass method** or **Durand–Kerner method**, discovered by Karl Weierstrass in 1891 and rediscovered independently by Durand in 1960 and Kerner in 1966, is a root-finding algorithm for solving polynomial equations.[1] In other words, the method can be used to solve numerically the equation $f(x)=0$, where f is a given polynomial, which can be taken to be scaled so that the leading coefficient is 1.

https://en.wikipedia.org/wiki/Durand%E2%80%93Kerner_method, accessed 20260313

Durand–Kerner Method

Explanation [edit]

This explanation considers equations of degree four. It is easily generalized to other degrees.

Let the polynomial *f* be defined by

$$f(x) = x^4 + ax^3 + bx^2 + cx + d$$

for all *x*. The known numbers *a*, *b*, *c*, *d* are the coefficients.

Let the (potentially complex) numbers *P*, *Q*, *R*, *S* be the roots of this polynomial *f*. Then

$$f(x) = (x - P)(x - Q)(x - R)(x - S)$$

for all *x*. One can isolate the value *P* from this equation:

$$P = x - \frac{f(x)}{(x - Q)(x - R)(x - S)}.$$

https://en.wikipedia.org/wiki/Durand%E2%80%93Kerner_method, accessed 20260313

Durand–Kerner Method

So if used as a fixed-point iteration

$$x_1 := x_0 - \frac{f(x_0)}{(x_0 - Q)(x_0 - R)(x_0 - S)},$$

it is strongly stable in that every initial point $x_0 \neq Q, R, S$ delivers after one iteration the root $P = x_1$. Furthermore, if one replaces the zeros Q, R and S by approximations $q \approx Q$, $r \approx R$, $s \approx S$, such that q, r, s are not equal to P, then P is still a fixed point of the perturbed fixed-point iteration

$$x_{k+1} := x_k - \frac{f(x_k)}{(x_k - q)(x_k - r)(x_k - s)},$$

since

$$P - \frac{f(P)}{(P - q)(P - r)(P - s)} = P - 0 = P.$$

Note that the denominator is still different from zero. This fixed-point iteration is a contraction mapping for x around P.

https://en.wikipedia.org/wiki/Durand%E2%80%93Kerner_method, accessed 20260313

Durand–Kerner Method

The clue to the method now is to combine the fixed-point iteration for P with similar iterations for Q, R, S into a simultaneous iteration for all roots.

Initialize p, q, r, s:

$$p_0 := (0.4 + 0.9i)^0,$$
$$q_0 := (0.4 + 0.9i)^1,$$
$$r_0 := (0.4 + 0.9i)^2,$$
$$s_0 := (0.4 + 0.9i)^3.$$

There is nothing special about choosing $0.4 + 0.9i$ except that it is neither a real number nor a root of unity.

https://en.wikipedia.org/wiki/Durand%E2%80%93Kerner_method, accessed 20260313

Durand–Kerner Method

Make the substitutions for n = 1, 2, 3, ...:

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{(p_{n-1} - q_{n-1})(p_{n-1} - r_{n-1})(p_{n-1} - s_{n-1})},$$
$$q_n = q_{n-1} - \frac{f(q_{n-1})}{(q_{n-1} - p_n)(q_{n-1} - r_{n-1})(q_{n-1} - s_{n-1})},$$
$$r_n = r_{n-1} - \frac{f(r_{n-1})}{(r_{n-1} - p_n)(r_{n-1} - q_n)(r_{n-1} - s_{n-1})},$$
$$s_n = s_{n-1} - \frac{f(s_{n-1})}{(s_{n-1} - p_n)(s_{n-1} - q_n)(s_{n-1} - r_n)}.$$

Re-iterate until the numbers p, q, r, s essentially stop changing relative to the desired precision. They then have the values P, Q, R, S in some order and in the chosen precision. So the problem is solved.

Note that complex number arithmetic must be used, and that the roots are found simultaneously rather than one at a time.

https://en.wikipedia.org/wiki/Durand%E2%80%93Kerner_method, accessed 20260313

Luisito L. Agustin

Roots of Polynomials as an Eigenvalue Problem

References

[Chapra] (book) Steven C. Chapra: *Applied Numerical Methods with MATLAB for Engineers and Scientists*, Int'l ed, 2005.

[Kreyszig] (book) Erwin Kreyszig: *Advanced Engineering Mathematics*, 10th ed, 2011.
(Part E Numeric Analysis, Chap 19-21).

Eigenvalues

The eigenvalues of $\mathbf{A}$ *are the solutions* λ *of the* **characteristic equation**

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \begin{vmatrix} a_{11} - \lambda & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} - \lambda & \cdots & a_{2n} \\ \cdot & \cdot & \cdots & \cdot \\ a_{n1} & a_{n2} & \cdots & a_{nn} - \lambda \end{vmatrix} = 0. \tag{3}$$

[Kreyszig], 20.6 Matrix Eigenvalue Problems: Introduction

Roots of Polynomials as an Eigenvalue Problem

Suppose we have a polynomial

$$a_1x^5 + a_2x^4 + a_3x^3 + a_4x^2 + a_5x + a_6 = 0 \tag{6.13}$$

Dividing by a_1 and rearranging yields

$$x^5 = -\frac{a_2}{a_1}x^4 - \frac{a_3}{a_1}x^3 - \frac{a_4}{a_1}x^2 - \frac{a_5}{a_1}x - \frac{a_6}{a_1}$$

A special matrix can be constructed by using the coefficients from the right-hand side as the first row and with 1's and 0's written for the other rows as shown:

$$\begin{bmatrix} -a_2/a_1 & -a_3/a_1 & -a_4/a_1 & -a_5/a_1 & -a_6/a_1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \tag{6.14}$$

Equation (6.14) is called the polynomial's *companion matrix*. It has the useful property that its eigenvalues are the roots of the polynomial. Thus, the algorithm underlying the `roots` function consists of merely setting up the companion matrix and then using MATLAB's powerful eigenvalue evaluation function to determine the roots.

[Chapra], 6.6 Polynomials